

PF COM LISP

Pedro Henrique Ribeiro Dias

Inatel

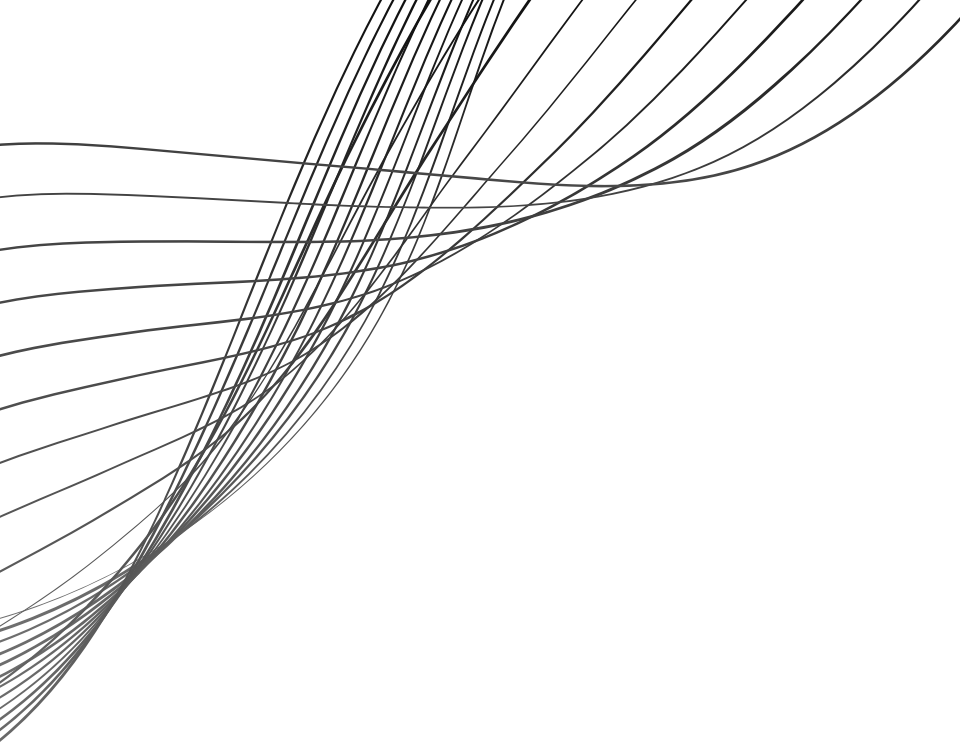
INTRODUÇÃO

LISP: A Linguagem que Inventou o Futuro

Se o Haskell foi construído sobre matemática, o LISP foi construído sobre uma ideia radical – código e dados são a mesma coisa. Criado em 1958 por John McCarthy, o LISP é a segunda linguagem de alto nível mais antiga ainda em uso, e praticamente toda linguagem funcional moderna deve algo a ele.

O que torna o LISP único?

- **Homoiconicidade:** Em LISP, o código é escrito na mesma estrutura que os dados – listas. Isso significa que um programa LISP pode gerar e manipular outros programas LISP como se fossem dados comuns, algo que poucas linguagens conseguem fazer de forma tão natural.
- **Parênteses como Estrutura:** A sintaxe do LISP é quase inexistente – tudo são listas aninhadas entre parênteses. O que parece estranho no início é na verdade a forma mais uniforme de representar código que já foi criada.
- **Macros Poderosas:** Diferente de outras linguagens, no LISP você pode estender a própria linguagem criando novas construções de sintaxe – algo que vai muito além do que funções comuns permitem.



INTRODUÇÃO

"Aprender LISP é entender de onde vieram as ideias que hoje chamamos de modernas. Garbage collection, funções de alta ordem, tipagem dinâmica – tudo começou aqui."

PILARES DA PF

Funções Puras: a saída depende apenas da entrada. A mesma chamada com os mesmos argumentos sempre retorna o mesmo resultado – sem modificar nada fora dela.

Imutabilidade: uma vez que um valor é definido, ele não muda. Isso elimina efeitos colaterais e torna o fluxo de dados muito mais fácil de acompanhar.

Expressões vs. Comandos: na PF não existem instruções que alteram estado – tudo é uma expressão que retorna um valor. Você descreve o que é, não o que fazer.

Composição de Funções: funções pequenas e simples são combinadas para formar comportamentos mais complexos, como blocos de montar.

A LINGUAGEM LISP

- Origem: Segunda linguagem de programação de alto nível (depois do Fortran), criada por John McCarthy (1958).
- LISP = LISt Processing: Tudo no código e nos dados é representado como listas.
- S-Expressions: A sintaxe é uniforme
- Em Lisp, a ênfase é em criar novos dados em vez de modificar os existentes.

A LINGUAGEM LISP

A principal característica do Lisp: o código e os dados têm a mesma estrutura (listas).

- Expressão: (+ 1 2)

Lista de Dados: '(1 2 3) (A aspas ' indica que é uma lista de dados, não código para ser executado.)

- Isso permite que programas escrevam ou manipulem outros programas.

EXERCÍCIO 1

Exercício 1 – Farmácia da Maomao

Maomao precisa calcular a dosagem e o preço das ervas que prepara para seus pacientes. Cada erva tem um preço base, e a dosagem varia de acordo com o peso e a idade de quem vai tomar.

- Defina um tipo `Erva` com os campos `nome` e `precoBase`.
- Implemente `calculaDosagem :: Double -> Int -> String` que recebe peso e idade e usa guards para retornar a dosagem como texto – "10ml" para crianças pequenas ou abaixo do peso, "25ml" para a faixa intermediária e "50ml" para os demais casos.
- Implemente `ajustaPreco :: Erva -> Double` que usa pattern matching no nome da erva para aplicar o fator de preço – "Ginseng" multiplica por 3.0, "Lotus" multiplica por 1.5 e qualquer outro retorna o preço base sem alteração.
- Implemente `descricaoErva :: Erva -> Double -> Int -> String` que recebe uma erva, o peso e a idade do paciente e retorna uma String com o nome da erva, o preço ajustado e a dosagem indicada.
- Na main, crie três ervas – "Ginseng", "Lotus" e "Camomila" – e exiba a descrição para pelo menos dois pacientes com idades e pesos diferentes, cobrindo todos os casos de dosagem.

EXERCÍCIO 2

Exercício 2 – Fauna de Planet 4546B

O Scanner de mão do Ryley está catalogando as criaturas encontradas em 4546B. Cada criatura registrada tem seu ambiente, nível de perigo e tempo de vida, e o sistema precisa filtrar e gerar relatórios sobre elas.

- Defina o tipo Criatura com os campos nome, ambiente, periculosidade e vidaMedia.
- Crie o tipo Periculosidade com os construtores Baixa, Media e Alta.
- Na main, defina uma lista catalogo com pelo menos quatro criaturas – um Peeper das Safe Shallows com periculosidade Baixa, um Reaper Leviathan com Alta, e duas criaturas do ambiente "Deep" à sua escolha.
- Implemente `filtraPorPerigo :: [Criatura] -> [Criatura]` usando filter para retornar apenas as criaturas cuja periculosidade não seja Baixa.

EXERCÍCIO 2

- Implemente `relatorioProfundidade :: [Criatura] -> [String]` que primeiro filtra só as criaturas do ambiente "Deep" e depois usa `map` para formatar cada uma como "[NOME]: Vive em [AMBIENTE]".
- Implemente `descricaoCriatura :: Criatura -> String` que retorna uma `String` com nome, ambiente e periculosidade da criatura, para ser usada na listagem geral.
- Na `main`, exiba o catálogo completo com `descricaoCriatura`, depois as criaturas perigosas filtradas e por fim o relatório de profundidade.

EXERCÍCIO 3

Exercício 3 – Loja Clandestina de Iwai

Você gerencia uma loja secreta que vende armas, poções e artefatos mágicos. Cada item tem um preço e uma força mágica, e os itens mais poderosos recebem um bônus especial na hora da venda.

- Defina o tipo `Tipoltem` com os construtores `Arma`, `Pocao` e `Artefato`.
- Defina o tipo `Item` com os campos `nome`, `tipo`, `preco` e `forcaMagica`.
- Implemente `adicionalImposto :: Double -> Double` que aumenta o preço em 15%.
- Implemente `bonusMaldicao :: Double -> Double` usando guards – se `forcaMagica` for maior que 80, multiplica por 1.5, caso contrário retorna o valor original.
- Implemente `descricaoVenda :: Item -> String` que recebe um item já processado e retorna uma `String` com o nome, o preço com imposto e a força mágica com bônus aplicado.
- Implemente `processaVenda :: [Item] -> [String]` que em sequência: filtra só os itens do tipo `Arma` com `filter`, aplica `adicionalImposto` no preço de cada um com `map`, aplica `bonusMaldicao` na força mágica com `map`, e por fim usa `descricaoVenda` para formatar o resultado de cada arma.
- Na `main`, crie um catálogo com pelo menos cinco itens misturando os três tipos, chame `processaVenda` e exiba o resultado.

EXERCÍCIO 4

Exercício 4 – Viajantes de Teyvat

O Viajante está catalogando os domínios e chefes de campo encontrados em Teyvat. Cada local tem um elemento associado, um nível de dificuldade e uma recompensa em mora. Nahida precisa ajudar a calcular qual rota de farm vale mais a pena.

- Defina o tipo Elemento com os construtores Pyro, Hydro, Cryo, Electro e Anemo.
- Defina o tipo Local com os campos nome, elemento, dificuldade e recompensa.
- Implemente `bonusElemental :: Local -> Double` usando guards – se o elemento for Pyro ou Electro aplica um bônus de 20% na recompensa, se for Cryo aplica 10%, e nos demais casos retorna a recompensa original.
- Implemente `valeAPena :: Local -> Bool` que retorna True se a recompensa com bônus for maior que 500 e a dificuldade for menor ou igual a 3.
- Implemente `rotaDeFarm :: [Local] -> [String]` que usa filter com `valeAPena` para selecionar os locais recomendados e map para formatar cada um como "[NOME] – Recompensa: [VALOR]".
- Na main, crie um catálogo com pelo menos cinco locais com elementos e dificuldades variados, exiba a lista completa com `bonusElemental` de cada um e em seguida exiba apenas a rota de farm recomendada.

FIM DA AULA!



zSh3rl0cK/S01-monitoria

Contribute to zSh3rl0cK/S01-monitoria development by creating an account on GitHub.

